Bound the Invisible: Gk-tree based Privacy-preserving Reverse Skyline Query

Mingfeng Jiang, Hua Dai, *Member, IEEE*, Rui Zhang, Huaqun Wang, Hao Zhou, Debiao He, *Member, IEEE*, Bohan Li

***Abstract*—With the rapid development of the data economy, data marketplaces have become important platforms for data trading and location-aware service provisioning. However, efficiently supporting product promotion, targeted advertising, and market analysis while preserving data privacy remains a significant challenge, especially in cloud-based environments where large volumes of queries are continuously generated by users. In this paper, we propose an efficient privacy-preserving reverse skyline query scheme to identify potential users that are most likely to be interested in a target product. We first propose the Global Nearest Neighbor based Dynamic Dominance Region (GDDR), which utilizes global nearest neighbor relationships to construct regularized dominance regions, thereby reducing the complexity of geometric boundary evaluation. Based on the GDDR, we further design the Gk-tree index by leveraging global spatial properties to support hierarchical pruning over encrypted data, which significantly improves query efficiency. Security analysis shows that the proposed scheme achieves IND-SCPA security. Experimental evaluations conducted on real world dataset demonstrate that the proposed scheme achieves substantially better query performance than existing schemes.**

***Index Terms*—data marketplace, searchable encryption, reverse skyline query, multi-dimensional data**

I. Introduction

With the rapid development of digital economy, data marketplaces have emerged as critical cloud-based platforms for data trading [1]–[3]. Among various information retrieval methods, the skyline query serves as a fundamental query strategy, enabling users to identify points of interest by selecting non-dominated objects from multi-dimensional datasets. Therefore, skyline queries have been widely applied in data mining, multi-dimensional decision-making, and other related fields [4]. With the proliferation of mobile computing and location-based services, massive volumes of spatial queries are continuously generated by mobile users through smart devices and edge networks. As a typical extension of skyline query, the reverse skyline query is widely applied in practical scenarios such as environmental monitoring and recommendation systems. Specifically, in data marketplaces, the reverse skyline query is applied to identify users who are likely to select the queried data based on their individual preferences [5]. By capturing such users, the reverse skyline query facilitates accurate matching between users and data resources, thereby supporting demand analysis and competitiveness evaluation. However, as data marketplaces increasingly migrate to outsourced cloud environments, the demand for privacy preservation has become paramount. Directly uploading large volumes of raw data to cloud platforms introduces significant risks of information leakage. Moreover, query requests contain sensitive attributes of users, including precise location information or personal preferences. Therefore, balancing privacy preservation and query efficiency remains a fundamental challenge in skyline query processing, especially in secure data trading environments [6], [7].

In this paper, we focus on designing an efficient and privacy-preserving reverse skyline query scheme for data marketplaces. To demonstrate its practical applicability, we consider a medical data marketplace in which a data provider publishes a novel clinical dataset. The dataset contains multiple attributes, such as diagnostic accuracy, sample size, acquisition cost, disease coverage, and update frequency. The provider aims to identify potential buyers from a collection of research institutions, hospitals, or pharmaceutical companies. Each institution expresses its requirements through multi-dimensional preference vectors according to its own research objectives, budget constraints, and analytical demands. By performing a reverse skyline query, the provider can efficiently identify institutions that regard the dataset as a competitive and non-dominated choice compared with other available datasets in the marketplace.

Such a query mechanism is highly valuable in practical data trading environments. Instead of blindly advertising datasets to all users, data providers can accurately identify potential buyers whose preferences are closely aligned with the characteristics of the published dataset. This significantly improves recommendation precision, reduces unnecessary communication overhead, and enhances the efficiency of data transactions. Moreover, reverse skyline queries also support market competitiveness analysis by enabling providers to evaluate the influence of their datasets relative to competing products under diverse user preferences.

However, despite these advantages, the query processing inevitably introduces serious privacy concerns. On the one hand, research institutions and medical organizations are often unwilling to disclose sensitive research interests, disease focuses, funding conditions, or analytical intentions, since such information may reveal strategic research directions or commercial plans. On the other hand, data providers must protect the commercial value of their high-quality clinical datasets from exposure to untrusted cloud servers. Directly

M. Jiang, H. Dai, R. Zhang, H. Wang, H. Zhou, are with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: 2024040413@njupt.edu.cn, daihua@njupt.edu.cn, 1758900915@qq.com, wanghuaqun@aliyun.com, haozhou@njupt.edu.cn).

D. He is with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: hedebiao@163.com).

B. Li is with the School of College of Artificial Intelligence, Nanjing University of Aeronautics and Astronautics, Nanjing 210023, China (e-mail: bhli@nuaa.edu.cn).

outsourcing raw datasets and query requests to the cloud may result in severe privacy leakage, including the exposure of sensitive medical attributes, pricing strategies, and user preference information.

Furthermore, in practical cloud-based data marketplaces, reverse skyline queries are usually executed continuously and at large scale. Therefore, an effective privacy-preserving reverse skyline query scheme should not only support accurate and efficient query execution, but also prevent the cloud server from learning sensitive data attributes, user preferences, and commercial intentions throughout the entire query processing.

Existing research has explored various approaches for reverse skyline query processing, yet several limitations remain. For instance, [8], [9] mainly focused on improving query efficiency in high dimensional spaces. However, their indexing structures may leak sensitive attribute information during query processing. [10] investigated privacy-preserving reverse skyline query, but the proposed solution relied on a two-cloud model, which introduced substantial communication overhead. To address this issue, [11] proposed a single-server scheme. Nevertheless, its query efficiency remained unsatisfactory when handling large scale datasets. Additionally, [12] attempted to improve query efficiency while mitigating access pattern leakage. However, the scheme adopted an index free design, resulting in a large amount of redundant ciphertext evaluations and significantly limiting scalability. Therefore, achieving high query efficiency through a secure index while preserving data privacy remains a critical and unresolved challenge.

To address the above challenges, we propose the Gk-tree based Privacy-preserving Reverse Skyline Query (GPRSQ) scheme in this paper. The main contributions are shown as follows:

1) We design the **Global Nearest Neighbor-based Dynamic Dominance Region (GDDR)**, which aims to transform complex dynamic dominance boundary evaluation into efficient distance vector comparison in encrypted domains. The proposed GDDR utilizes the global nearest neighbor to construct a regularized dominance sub-region for each data vector, enabling privacy-preserving reverse skyline query without revealing sensitive data attributes or user preferences.
2) By integrating the geometric properties of GDDR into a kd-tree, we design the **GDDR-based kd-tree Index (Gk-tree)**, which supports hierarchical pruning. By combining GDDR-based pruning boundaries within the tree nodes, the cloud server can discard large volumes of irrelevant data vectors at the internal node level. This integration transforms the reverse skyline query processing from linear scan into a sub-linear query, significantly enhancing the query efficiency.
3) Based on the GDDR and Gk-tree index, we propose the **Gk-tree based Privacy-preserving Reverse Skyline Query** (GPRSQ). The proposed scheme ensures that the cloud server remains oblivious to the data vectors, query requests, and final query results, thereby providing privacy preservation together with high query efficiency.
4) We provide formal correctness and security analysis for the proposed scheme, and prove that it achieves IND-SCPA security. In addition, we conduct comprehensive experiments on real world dataset to evaluate the effectiveness and efficiency of the proposed scheme.

The remainder of this paper is organized as follows: Section II describes the related work; Section III provides the preliminaries, including the notations and problem formulation; Section IV describes the definition and security model; Section V proposes the GPRSQ scheme; Section VI gives the security analysis; Section VII shows the performance evaluation; and the last section concludes the paper.

II. Related Work

A. Skyline Query

The concept of the static skyline query was first introduced by Börzsönyi et al. [13] to extract globally optimal objects from a multi-dimensional dataset. Building on this, Papadias et al. [14] formulated the dynamic skyline query by evaluating dominance relative to a specific query data vector. Subsequently, Dellis and Seeger [15] inverted this user-centric logic to assess the influence of data vectors on users, formally introducing the reverse skyline query.

To improve the execution efficiency of static skyline queries, recent research has shifted from basic enumeration algorithms (e.g., BNL and BBS) to system-level optimizations and semantic enhancements. Miao et al. [16] proposed an effective cardinality estimation method for the skyline family, enabling query optimizers to accurately predict output sizes and accelerate execution. To mitigate the inherent defect of uncontrollable output sizes, Mouratidis et al. [17] and Ciaccia et al. [18] explored the integration of skyline and top-k queries, building upon flexible preference frameworks [19] to achieve ranked, size-controllable results efficiently.

For dynamic skyline queries, the primary efficiency bottleneck lies in the continuous multi-dimensional distance comparisons. To address this, recent studies focus on spatial precomputation and continuous monitoring. Liu et al. [20] proposed the Skyline Diagram, leveraging space partitioning to group query data vectors with identical dynamic skylines, thus avoiding repetitive on-the-fly evaluations. In highly dynamic scenarios, Hidayat et al. [21] developed efficient safe-zone techniques for the continuous monitoring of moving skyline queries, reducing the computational overhead.

Unlike dynamic queries, reverse skyline queries require global cross-referencing, making computational efficiency the primary bottleneck. Early foundational algorithms, such as the DDR-based BBRS scheme [22], suffer from severe overhead during full-table verification. To mitigate this, subsequent studies optimized pruning and indexing: Vlachou et al. introduced half-space pruning, while Zhu et al. [23] proposed a bitmap index for moving objects. For high-dimensional and combinatorial processing, Bian et al. [24] developed the QSRP scheme using score table sampling instead of tree pruning, and Li et al. [25] proposed a QR-GMap index to address multi-point combinatorial decisions. However, these plaintext schemes deeply depend on exposing spatial attributes and

exhaustive geometric comparisons, making their direct migration to privacy-preserving ciphertext environments highly challenging.

B. Privacy-Preserving Skyline Query

When outsourcing skyline queries to the cloud, the evaluation of dominance relationships in the ciphertext domain faces a significant computational bottleneck due to its nonlinear characteristics. While continuously enhancing security strength, existing research has gradually shifted its focus toward overcoming the challenges of retrieval efficiency and communication overhead in complex queries.

Early privacy-preserving static skyline queries primarily relied on public-key or matrix encryption. For instance, Zheng et al. [26] inevitably leaked single-dimensional privacy. The schemes by Liu et al. [27] and Zhang et al. [28] suffered from high computational overhead in high dimensions and restricted numerical ranges due to bitmap representations, respectively. To enhance single-cloud security, Zhang et al. [29] adopted Symmetric Homomorphic Encryption (SHE) under a dual-cloud model to reduce communication. However, the fixed coordinate origin of static queries limits their applicability. To address this, dynamic skyline queries have emerged but face a severe efficiency-security trade-off. Liu et al. [30] achieved comprehensive privacy at the cost of protocol efficiency. Conversely, Wang et al. [31] and Zeighami et al. [32] utilized order-preserving encryption to boost speed, yet incurred inference risks by leaking single-dimensional partial orders.

Unlike dynamic queries, reverse skyline queries require global cross-references to determine dominance, making computational efficiency the primary bottleneck. To address this, Zhang et al. [10] proposed a dual-cloud OPPRS scheme using Paillier encryption; however, its performance is severely constrained by expensive ciphertext multiplications and heavy communication. Subsequently, Zhang et al. [11] developed a single-cloud PPARS scheme based on fully homomorphic encryption, but it relies on a full-table linear scan and leaks partial plaintext to gain usability. Recently, Peng et al. [12] introduced a single-cloud ePRSQ scheme using symmetric homomorphic encryption and vector inner products. Although it reduces cryptographic overhead, it still requires global $O(n^2)$ ciphertext comparisons to prevent access pattern leakage, structural-limiting its retrieval efficiency.

In summary, to ensure security, existing privacy-preserving reverse skyline query schemes adopt strategies such as the two-cloud model and pairwise ciphertext comparisons, which constrain their scalability on large-scale datasets. To further improve efficiency, this paper proposes a novel scheme featuring a hierarchical tree index that enables sub-linear query pruning within the ciphertext domain.

III. Preliminaries

A. Notations

The notations used in this paper are summarized in Table I, presented in the order in which they appear throughout the scheme. It is noticeable that bold symbols such as $\mathbf{u} = (\mathbf{u}[0], \mathbf{u}[1], \cdots, \mathbf{u}[n-1])$ denote vectors, where $\mathbf{u}[i]$ represents the i-th component of the vector. Moreover, given a matrix $M \in \mathbb{Z}_q^{n\times m}$, $M[j]$ represents the j-th column of the matrix.

TABLE I: Notations

Notations	Descriptions
D	The multi-dimensional dataset
$\mathbf{p}_i$	The data vector
$\mathbf{q}$	The query data vector
$RS_{\mathbf{q}}$	The reverse skyline result set of $\mathbf{q}$
$\mathbf{o}_i$	The global nearest neighbor of $\mathbf{p}_i$
$\boldsymbol{\rho}_i$	The GDDR radius of $\mathbf{p}_i$
D_ρ	The GDDR radius set of D
$\hat{R}_i$	The GDDR of $\mathbf{p}_i$
$\mathcal{T}$	The Gk-tree index
α	The splitting dimension
$\mathbf{c}_m$	The virtual geometric center of the MBR
$\boldsymbol{\rho}_a$	The aggregated GDDR radius
$\mathbf{o}_a$	The aggregated boundary point
$\mathbf{p}_r$	The exact vector
tk	The query token
C	The candidate set

B. k-dimension Tree

The k-dimension tree (kd-tree) [33] is a hierarchical indexing structure for organizing a d-dimensional point set $D = \{\mathbf{p}_1, \mathbf{p}_2, \cdots, \mathbf{p}_n\}$. Specifically, the kd-tree decomposes the original space S into a series of non-overlapping hyper-rectangular regions by recursively selecting specific dimensions and utilizing hyperplanes for splitting at each level. During the construction of the kd-tree, each internal node N is associated with a splitting vector (j, v), where $j \in \{1, 2, \cdots, d\}$ denotes the splitting dimension and $v \in \mathbb{Z}$ represents the splitting value (typically the median along that dimension). For the point set D_N covered by the current node, the partitioning process follows the mapping logic where $D_L = \{\mathbf{p}_i \in D_N \mid \mathbf{p}_i[j] \le v\}$ and $D_R = \{\mathbf{p}_i \in D_N \mid \mathbf{p}_i[j] > v\}$. Through this recursive decomposition, points are assigned to the left subtree T_L and the right subtree T_R until a leaf node termination criterion is met. To ensure query efficiency, a weight-balanced mechanism is typically introduced during construction. Given a regulation factor $\alpha \in [0, 0.5)$, for any node N and its corresponding point set size $|D_N|$, the sizes of its left and right subtrees satisfy that $\max(|D_L|, |D_R|) \le (0.5+\alpha)|D_N|$, which ensures that the tree depth is maintained at the $O(\log_{1/(0.5+\alpha)} n)$ scale.

C. Dynamic Skyline Query

In a d-dimensional space, the dynamic skyline query aims to identify the optimal candidate set that best satisfies the preferences for a specific query data vector $\mathbf{q}$. The core mechanism involves evaluating the dominance relationship among the points in the dataset by using $\mathbf{q}$ as a dynamic reference point, which requires comparing the distance differences from each data vector to $\mathbf{q}$ across all dimensions.

During the computation, this evaluation relies on the dynamic dominance relationship: if a data vector is dynamically dominated by another point in the dataset, it implies that there is a comprehensively better alternative for satisfying the query data vector $\mathbf{q}$, and thus the dominated point is pruned. After a global comparison, only the data vectors that are

not dynamically dominated by any other point are ultimately included in the dynamic skyline set. The formal definitions for dynamic dominance and the dynamic skyline query are provided below:

Definition 1. *Dynamic Dominance:* Given a reference point $\mathbf{q}$ and two data vectors $\mathbf{p}_a, \mathbf{p}_b \in D$, if Eq. 1 satisfies, then $\mathbf{p}_a$ is said to dynamically dominate $\mathbf{p}_b$ with respect to $\mathbf{q}$, denoted as $\mathbf{p}_a \prec_{\mathbf{q}} \mathbf{p}_b$.

$$\begin{cases} |\mathbf{p}_a[i] - \mathbf{q}[i]| \leq |\mathbf{p}_b[i] - \mathbf{q}[i]|, & \forall i \in \{1, 2, \cdots, d\} \\ |\mathbf{p}_a[j] - \mathbf{q}[j]| < |\mathbf{p}_b[j] - \mathbf{q}[j]|, & \exists j \in \{1, 2, \cdots, d\} \end{cases} \tag{1}$$

Definition 2. *Dynamic Skyline Query:* Given a dataset D and a query data vector $\mathbf{q}$, the dynamic skyline point set of $\mathbf{q}$, denoted as $DS_{\mathbf{q}}$, comprises the subset of points in D that are not dynamically dominated by any other point with respect to $\mathbf{q}$. Formally, $DS_{\mathbf{q}}$ is defined as:

$$DS_{\mathbf{q}} = \{\mathbf{p}_i \in D \mid \nexists \mathbf{p}'_i \in D, \mathbf{p}'_i \prec_{\mathbf{q}} \mathbf{p}_i\}.$$

D. Dynamic Dominance Region

The Dynamic Dominance Region (DDR) is defined as a spatial subset characterized by an absolute disadvantage in dynamic comparisons relative to a reference data vector $\mathbf{p}_i$, due to the presence of the dynamic skyline points associated with $\mathbf{p}_i$. The primary function of this region is to define a pruning zone during the computation process, thereby facilitating the rapid filtration of unqualified candidate points.

Specifically, given a reference data vector $\mathbf{p}_i \in \mathbb{Z}^d$ and its corresponding dynamic skyline point set $DS_{\mathbf{p}_i}$, the relative distance vector of a point $\mathbf{s}_i \in DS_{\mathbf{p}_i}$ with respect to $\mathbf{p}_i$ is denoted as $\mathbf{v}_i = (|\mathbf{s}[1] - \mathbf{p}[1]|, |\mathbf{s}[2] - \mathbf{p}[2]|, \cdots, |\mathbf{s}[d] - \mathbf{p}[d]|)$. According to the definition of dynamic dominance, the single-point dynamic dominance region, denoted as SR_i, is formed by $\mathbf{s}_i$ partitioning a sub-region in the space that is absolutely inferior to itself relative to $\mathbf{p}_i$.

For any point in the space, such as a query data vector $\mathbf{q}$, to fall into this sub-region, its distance to $\mathbf{p}_i$ must be greater than or equal to the distance from $\mathbf{s}_i$ to $\mathbf{p}_i$ across all dimensions, and strictly greater in at least one dimension, i.e., $SR_i = \{\mathbf{q} \in \mathbb{Z}^d \mid \forall i \in \{1, 2, \cdots, d\}, |\mathbf{q}_i - \mathbf{p}_i| \geq |\mathbf{s}_i - \mathbf{p}_i| \wedge \exists j \in \{1, 2, \cdots, d\}, |\mathbf{q}_j - \mathbf{p}_j| > |\mathbf{s}_j - \mathbf{p}_j|\}$. In the transformed distance space where $\mathbf{p}_i$ serves as the origin, SR_i is defined as an unbounded hyper-rectangle region extending infinitely outward with the relative distance vector $\mathbf{v}_i$ as its vertex.

Within this framework, the dynamic skyline point set $DS_{\mathbf{p}_i}$ comprises a collection of mutually non-dominating points. Any point in the space, such as $\mathbf{q}$, loses its competitiveness if it is dynamically dominated by at least one skyline point within this set relative to $\mathbf{p}_i$. Consequently, the complete dynamic dominance region R of the data vector $\mathbf{p}_i$ is strictly equivalent to the union of all single-point dynamic dominance sub-regions, defined as $R_i = \bigcup_{\mathbf{s}_i \in DS_{\mathbf{p}_i}} SR_i$.

E. Asymmetric Matrix Encryption

Asymmetric Matrix Encryption (AME) [34] is an asymmetric encryption scheme designed to securely perform Euclidean distance computations and distance comparisons over encrypted data. Compared with the existing Asymmetric Scalar-Product-Preserving Encryption (ASPE) methods, the proposed AME introduces robust randomization and matrix transformations to ensure known-plaintext attacks (KPA) resistance. Let x_1, x_2 denote two d-dimensional points and q is a d-dimensional query data vector, respectively. The secret key $sk = \{\hat{M}_L^{e,f,g}, \hat{M}_R^{e,f,g}, \tilde{M}_L^{e,f,g}, \tilde{M}_R^{e,f,g}\}_{e,f,g=1}^2$ consists of a collection of random $(2d+6)\times(2d+6)$-dimensional invertible matrices. To facilitate distance comparison, the points are first extended into $(2d+2)$-dimensional vectors $\mathbf{v}_{1,L}$, $\mathbf{v}_{2,R}$ and a $(2d+2)\times(2d+2)$-dimensional matrix Q. By incorporating sets of random numbers, they are further expanded into $(2d+6)$-dimensional vectors $\hat{\mathbf{v}}_{1,L}, \tilde{\mathbf{v}}_{1,L}, \hat{\mathbf{v}}_{2,R}, \tilde{\mathbf{v}}_{2,R}$ and $(2d+6)\times(2d+6)$-dimensional matrices $\hat{Q}, \tilde{Q}$. These vectors and matrices are additively split into two shares as follows:

$$\begin{cases} \hat{Q}_1 + \hat{Q}_2 = \hat{Q} \\ \hat{\mathbf{v}}_{1,L,1} + \hat{\mathbf{v}}_{1,L,2} = \hat{\mathbf{v}}_{1,L} \end{cases}$$

Thus we have that

$$\begin{aligned} \hat{\mathbf{v}}_{1,L}^{e,f,g} &= \hat{\mathbf{v}}_{1,L,e}^{\mathrm{T}} \hat{M}_L^{e,f,g} \\ \hat{M}_q^{e,f,g} &= (\hat{M}_L^{e,f,g})^{-1} \hat{Q}_f \hat{M}_R^{e,f,g} \\ \hat{\mathbf{v}}_{2,R}^{e,f,g} &= (\hat{M}_R^{e,f,g})^{-1} \hat{\mathbf{v}}_{2,R,g} \end{aligned}$$

where $e, f, g \in \{1, 2\}$. Similarly, we can compute $\tilde{\mathbf{v}}_{1,L}$, $\tilde{Q}$, and $\tilde{\mathbf{v}}_{2,R}$. Thus we have the encrypted vectors sets $\mathcal{V}_{1,L} = \{\hat{\mathbf{v}}_{1,L}^{e,f,g}, \tilde{\mathbf{v}}_{1,L}^{e,f,g}\}_{e,f,g=1}^2$, $\mathcal{V}_{2,R} = \{\hat{\mathbf{v}}_{2,R}^{e,f,g}, \tilde{\mathbf{v}}_{2,R}^{e,f,g}\}_{e,f,g=1}^2$ and matrices sets $\mathcal{M}_Q = \{\hat{M}_q^{e,f,g}, \tilde{M}_q^{e,f,g}\}_{e,f,g=1}^2$. To facilitate distance comparisons over encrypted data, AME defines an evaluation operator, denoted as $\mathsf{AmeEval}(\cdot)$. Given $\mathcal{V}_{1,L}$, $\mathcal{M}_Q$, and $\mathcal{V}_{2,R}$, we have that:

$$\begin{aligned} &\mathsf{AmeEval}(\mathcal{V}_{1,L}, \mathcal{M}_Q, \mathcal{V}_{2,R}) \\ =& \sum_{e,f,g=1}^{2} (\hat{\mathbf{v}}_{1,L}^{e,f,g} \hat{M}_q^{e,f,g} \hat{\mathbf{v}}_{2,R}^{e,f,g} + \tilde{\mathbf{v}}_{1,L}^{e,f,g} \tilde{M}_q^{e,f,g} \tilde{\mathbf{v}}_{2,R}^{e,f,g}) \\ =& \hat{\mathbf{v}}_{1,L}^{\mathrm{T}} \hat{Q} \hat{\mathbf{v}}_{2,R} + \tilde{\mathbf{v}}_{1,L}^{\mathrm{T}} \tilde{Q} \tilde{\mathbf{v}}_{2,R} \\ =& res \end{aligned}$$

Therefore, we can deduce that $res > 0$ is equivalent to $\mathbf{v}_{1,L}^{\mathrm{T}} Q \mathbf{v}_{2,R} \geq 0$, thus $\mathrm{dist}(x_1, q) \leq \mathrm{dist}(x_2, q)$.

F. Problem Formulation

Definition 3. *Multi-dimensional Dataset:* Let $D = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_n\}$ be a multi-dimensional dataset consisting of n data vectors in a d-dimensional space. Each data vector $\mathbf{p}_i \in D$ is represented as a d-dimensional vector $\mathbf{p}_i = (\mathbf{p}_i[1], \mathbf{p}_i[2], \ldots, \mathbf{p}_i[d])$, where $\mathbf{p}_i[j] \in \mathbb{Z}_q, j \in [1, d]$.

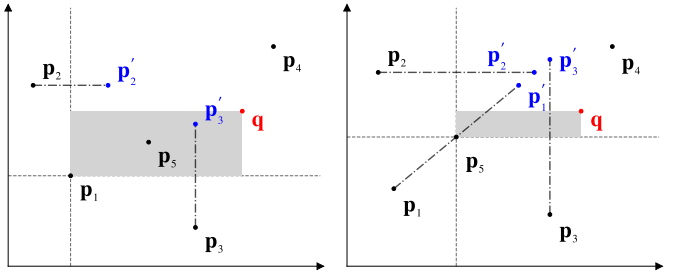
Definition 4. *Reverse Dominance:* Given a d-dimensional query data vector $\mathbf{q}$ and two data vectors $\mathbf{p}_a, \mathbf{p}_b \in D$, $\mathbf{p}_b$ is said to reverse dominate $\mathbf{q}$ with respect to $\mathbf{p}_a$, denoted as $\mathbf{p}_b \prec_{\mathbf{p}_a} \mathbf{q}$, if and only if the absolute distance from $\mathbf{p}_b$ to $\mathbf{p}_a$ is less than or equal to the distance from $\mathbf{q}$ to $\mathbf{p}_a$ in all dimensions, and strictly less in at least one dimension. Formally, the data vectors must satisfy the following conditions:

$$\begin{cases} |\mathbf{p}_b[i] - \mathbf{p}_a[i]| \leq |\mathbf{q}[i] - \mathbf{p}_a[i]|, & \forall i \in \{1, 2, \cdots, d\} \\ |\mathbf{p}_b[j] - \mathbf{p}_a[j]| < |\mathbf{q}[j] - \mathbf{p}_a[j]|, & \exists j \in \{1, 2, \cdots, d\} \end{cases}$$

Definition 5. *Reverse Skyline Query:* Given a multi-dimensional dataset D and a d-dimensional query data vector $\mathbf{q} \in \mathbb{Z}^d$, a point $\mathbf{p}_a \in D$ is considered a reverse skyline point of $\mathbf{q}$ if and only if $\mathbf{q}$ is not reverse dominated by any other point $\mathbf{p}_b \in D$ with respect to $\mathbf{p}_a$. A reverse skyline query retrieves the result set $RS_{\mathbf{q}}$ containing all such reverse skyline points in D. Formally, $RS_{\mathbf{q}}$ is defined as:

$$RS_{\mathbf{q}} = \{\mathbf{p}_a \in D \mid \nexists \mathbf{p}_b \in D, \mathbf{p}_b \prec_{\mathbf{p}_a} \mathbf{q}\}.$$

Fig. 1 illustrates an example of the reverse skyline query process, where all data vectors in the dataset $D = \{\mathbf{p}_i \mid 1 \leq i \leq 5\}$ and the query vector $\mathbf{q}$ are two-dimensional, represented as points in a two-dimensional coordinate system. Figs. 1(a) and 1(b) depict the reverse dominance relationships with respect to $\mathbf{p}_1$ and $\mathbf{p}_5$, respectively. To compare the values, all data vectors in D are mapped to the regions where $\mathbf{p}_1$ and $\mathbf{p}_5$ serve as the origins, and the rectangular areas constructed by the respective pivot points and $\mathbf{q}$ are marked in the figures. If a data vector falls within the rectangular area, it implies that this vector dominates $\mathbf{q}$ with respect to the origin point. As shown in Fig. 1(a), the data vector $\mathbf{p}_5$ falls inside the rectangle formed by $\mathbf{p}_1$ and $\mathbf{q}$, meaning that $\mathbf{p}_5$ dynamic-dominates $\mathbf{q}$, i.e., $\mathbf{p}_5 \prec_{\mathbf{p}_1} \mathbf{q}$. Thus, $\mathbf{p}_1$ is not a reverse skyline point. In contrast, as shown in Fig. 1(b), no data vector falls within the rectangle constructed by $\mathbf{p}_5$ and $\mathbf{q}$, indicating that no data vector in D dominates $\mathbf{q}$ with respect to $\mathbf{p}_5$. Therefore, $\mathbf{p}_5$ is a reverse skyline point. By evaluating all data vectors in D, we obtain the final reverse skyline query results as $RS_{\mathbf{q}} = \{\mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4, \mathbf{p}_5\}$.



(a) $\mathbf{p}$ is not a reverse skyline point. (b) $\mathbf{p}$ is a valid reverse skyline point.

Fig. 1: An example of reverse skyline query

IV. Definition and Security Model

A. System Model

Fig. 2 shows the system model of the proposed scheme GPRSQ, which consists of three components.

- **Data Owner (DO):** DO is responsible for the pre-processing of the dataset. Specifically, the DO constructs the index, encrypts the stored data, and subsequently outsources the encrypted index and datasets to CS.
- **Cloud Server (CS):** CS provides reliable data services and execute query processing. CS is assumed to be honest but curious, which implies that while the CS strictly adheres to the prescribed algorithms, it may also attempt to infer sensitive information from the stored datasets, the incoming query requests, and the corresponding results.
- **Data User (DU):** DU generates a token based on the query data vector, and then sends this token to CS to perform the reverse skyline query.

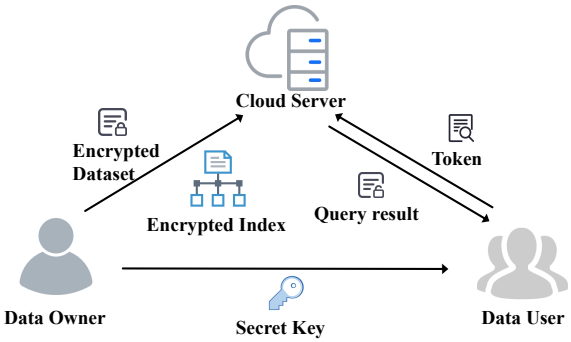


Fig. 2: System model

B. Definition

- $\mathsf{GenKey}(1^\lambda) \rightarrow sk$: Given the security parameter λ, this algorithm is executed by the DO to generate the secret key set sk.
- $\mathsf{BuildIndex}(D, sk) \rightarrow \widetilde{\mathcal{T}}$: Given the multi-dimensional dataset D and the secret key sk, this algorithm is executed by the DO. DO first generates the GDDR radius set D_ρ and constructs the Gk-tree index. Then DO encrypts the index as $\widetilde{\mathcal{T}}$ and sends it to CS.
- $\mathsf{GenToken}(\mathbf{q}, sk) \rightarrow tk$: Given the query data vector $\mathbf{q}$ and the secret key sk, this algorithm is executed by DU to generate the query token tk.
- $\mathsf{RSQuery}(\widetilde{\mathcal{T}}, tk) \rightarrow RS_{\mathbf{q}}$: Given the encrypted G$k$-tree index $\widetilde{\mathcal{T}}$ and the query token tk, this algorithm is executed by the CS. CS performs the pruning phase to generate the candidate set. Then CS conducts exact verification on the candidate set and outputs the query result $RS_{\mathbf{q}}$.

C. Leakage Function

In searchable encryption, the formal characterization of information disclosure is crucial for assessing security. While the proposed scheme ensures data confidentiality, certain knowledge must be revealed to CS to facilitate efficient query execution. Beyond the public security parameter λ, the information accessible to a curious CS typically includes the dataset size, query frequency, and result identifiers. Let $\Pi = (\mathsf{KeyGen}, \mathsf{BuildIndex}, \mathsf{GenToken}, \mathsf{RSQuery})$ denote our proposed scheme. Following the established security frameworks in [35]–[37], the leakage function $\mathcal{L}$ is defined as follows:

Definition 6. *Leakage Function:* As an essential component of the security definition in searchable encryption, we define a leakage function $\mathcal{L}$, which captures all potential information leaked to the CS during the evaluation of encrypted data under the proposed scheme Π. Given a multi-dimensional dataset D and a query request Q, the leakage function is expressed as $\mathcal{L}(D, Q)$, and includes:

The leakage function $\mathcal{L}(D, Q)$ captures all information exposed to CS during the interaction between a dataset D and a sequence of queries $Q = (q_1, q_2, \cdots, q_m)$. Specifically, $\mathcal{L}$ consists of the following two components:

- *Size Pattern*: This leakage reveals the global scale of the scheme, including the total number of data vectors in D and the count of query tokens issued by DUs.
- *Access Pattern*: This leakage refers to the set of identifiers associated with the encrypted records that satisfy each reverse skyline query.

D. Security Model

We utilize the game simulation to prove that the proposed scheme can be rigorously validated with Selective Chosen-Plaintext Attacks (IND-SCPA). We first establish the IND-SCPA game between the challenger $\mathcal{C}$ and the probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ to delineate the security model of our scheme. Within this security framework, we separately formalize IND-SCPA ciphertext indistinguishability and token unlinkability separately.

• *IND-SCPA ciphertext indistinguishability*

Init. $\mathcal{A}$ generates two multi-dimensional datasets D_0, D_1 with the same dimension and sends them to $\mathcal{C}$.

Setup. Given the secure parameter λ, $\mathcal{C}$ runs the GenKey algorithm to generate the secret key sk.

Phase 1. $\mathcal{A}$ issues multiple private-key queries in this phase. $\mathcal{C}$ responds in the following ways:

- $\mathsf{BuildIndex}$: $\mathcal{A}$ generates a multi-dimensional dataset D_i and send it to $\mathcal{C}$, $\mathcal{C}$ runs $\mathsf{BuildIndex}$ to generate the encrypted index $\widetilde{\mathcal{T}}$ and sends it back to $\mathcal{A}$.
- $\mathsf{GenToken}$: Given a query request Q_i, $\mathcal{C}$ runs $\mathsf{GenToken}$ to generate the token tk_i and sends it back to $\mathcal{A}$, where Q_i is subjected to $\mathcal{L}(D_0, Q_i) = \mathcal{L}(D_1, Q_i)$.

Challenge. With the datasets D_0 and D_1 generated in *Init*, $\mathcal{C}$ chooses a random bit $b \in \{0, 1\}$ and sets $C_b = \mathsf{BuildIndex}(D_b)$. It sends C_b as the challenge to $\mathcal{A}$.

Phase 2. $\mathcal{A}$ issues queries by the same constraints as in Phase 1. $\mathcal{C}$ responds as in Phase 1.

Guess. Finally, $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ and wins the game if $b' = b$.

The advantage of $\mathcal{A}$ to win the game is

$$Adv_{\Pi_{cipher},\mathcal{A}}(\lambda) = \left| Pr[b = b'] - \frac{1}{2} \right|.$$

Definition 7. *Ciphertext-IND-SCPA:* We assume that the scheme GPRSQ is Ciphertext-IND-SCPA secure if for any probabilistic polynomial-time IND-SCPA adversary $\mathcal{A}$, the function $Adv_{\Pi_{cipher},\mathcal{A}}(\lambda)$ is negligible.

• *IND-SCPA token unlinkability*

Init. $\mathcal{A}$ generates two query requests Q_0, Q_1 with the same dimension and sends them to $\mathcal{C}$.

Setup. Given the secure parameter λ, $\mathcal{C}$ runs the GenKey algorithm to generate the secret key sk.

Phase 1. $\mathcal{A}$ issues multiple private-key queries in this phase. $\mathcal{C}$ responds in the following ways:

- $\mathsf{BuildIndex}$: Given a multi-dimensional dataset D_i, $\mathcal{C}$ runs $\mathsf{BuildIndex}$ to generate the encrypted index $\widetilde{\mathcal{T}}$ and sends it back to $\mathcal{A}$, where D_i is subjected to $\mathcal{L}(D_i, Q_0) = \mathcal{L}(D_i, Q_1)$.
- $\mathsf{GenToken}$: Given a query request Q_i, $\mathcal{C}$ runs $\mathsf{GenToken}$ to generate the token tk_i and sends it back to $\mathcal{A}$.

Challenge. With the query requests Q_0, Q_1 generated in *Init*, $\mathcal{C}$ chooses a random bit $b \in \{0, 1\}$ and sets $tk_b = \mathsf{GenToken}(Q_b)$. It sends tk_b as the challenge to $\mathcal{A}$.

Phase 2. $\mathcal{A}$ issues queries by the same constraints as in Phase 1. $\mathcal{C}$ responds as in Phase 1.

Guess. Finally, $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ and wins the game if $b' = b$.

The advantage of $\mathcal{A}$ to win the game is

$$Adv_{\Pi_{token},\mathcal{A}}(\lambda) = \left| Pr[b = b'] - \frac{1}{2} \right|.$$

Definition 8. *Token-IND-SCPA:* We assume that the scheme GPRSQ is Token-IND-SCPA secure if for any polynomial time IND-SCPA adversary $\mathcal{A}$, the function $Adv_{\Pi_{token},\mathcal{A}}(\lambda)$ is negligible.

V. The Proposed Scheme

A. Global Nearest Neighbor-based Dynamic Dominance Region

In privacy-preserving scenarios, exact Dynamic Dominance Region (DDR) boundaries are irregular and non-convex, rendering the construction over ciphertext computationally prohibitive. Conventional exhaustive evaluation methods suffer from massive geometric computation overhead, which limits query scalability. To solve these challenges, we propose the Global Nearest Neighbor-based Dynamic Dominance Region (GDDR) to construct a regularly shaped sub-region of the DDR. By leveraging the global nearest neighbor of each data vector to approximate the dominance domain, the proposed GDDR effectively transforms intricate geometric boundary checks into distance vector comparisons, significantly enhancing query performance. The details are shown as follows.

Construction of GDDR. To construct the GDDR, we first generate the GDDR radius. Given the multi-dimensional dataset D, the global nearest neighbor of each data vector $\mathbf{p}_i \in D$, denoted as $\mathbf{o}_i$, is precomputed through the traversal of D. Consequently, the GDDR radius of $\mathbf{p}_i$, denoted as $\boldsymbol{\rho}_i \in \mathbb{Z}^d$, is calculated as follows:

$$\boldsymbol{\rho}_i[j] = |\mathbf{o}_i[j] - \mathbf{p}_i[j]|, \quad j \in \{1, 2, \cdots, d\},$$

and the GDDR radius set of D is $D_\rho = \{\boldsymbol{\rho}_1, \boldsymbol{\rho}_2, \cdots, \boldsymbol{\rho}_n\}$.

Based on the GDDR radius set D_ρ, the GDDR of $\mathbf{p}_i$, denoted as $\hat{R}_i$, is defined as a regular hyper-rectangle. Specifically, for any data vector $\mathbf{p}'$, it falls into $\hat{R}_i$ if its distance to $\mathbf{p}_i$ is greater than or equal to $\boldsymbol{\rho}_i$ across all dimensions, and strictly greater in at least one dimension. This condition can be formalized as follows:

$$\begin{cases} |\mathbf{p}'[j] - \mathbf{p}_i[j]| \geq \boldsymbol{\rho}_i[j], & \forall j \in \{1, 2, \cdots, d\} \\ |\mathbf{p}'[k] - \mathbf{p}_i[k]| > \boldsymbol{\rho}_i[k], & \exists k \in \{1, 2, \cdots, d\} \end{cases}$$

and the GDDR of $\mathbf{p}_i$ is $\hat{R}_i = \{\mathbf{p}' \mid \mathbf{o}_i \prec_{\mathbf{p}_i} \mathbf{p}'\}$. Consequently, $\hat{R}_i$ of $\mathbf{p}_i$ can be represented by $\mathbf{p}_i$, $\boldsymbol{\rho}_i$, and $\mathbf{o}_i$.

Lemma 1: For any data vector $\mathbf{p}_i \in D$, its global nearest neighbor $\mathbf{o}_i$ based on Euclidean distance is guaranteed to be a dynamic skyline point of $\mathbf{p}_i$, i.e., $\mathbf{o}_i \in DS_{\mathbf{p}_i}$.

PROOF: Assuming that $\mathbf{o}_i \notin DS_{\mathbf{p}_i}$, according to Definition 2, there must exist another point $\mathbf{s}_i \in D$ that dynamically dominates $\mathbf{o}_i$ with respect to $\mathbf{p}_i$, i.e., $\mathbf{s}_i \prec_{\mathbf{p}_i} \mathbf{o}_i$. According to Definition 1, $\mathbf{s}_i$ and $\mathbf{o}_i$ must satisfy:

$$\begin{cases} |\mathbf{s}_i[j] - \mathbf{p}_i[j]| \le |\mathbf{o}_i[j] - \mathbf{p}_i[j]|, & \forall j \in \{1, 2, \cdots, d\} \\ |\mathbf{s}_i[k] - \mathbf{p}_i[k]| < |\mathbf{o}_i[k] - \mathbf{p}_i[k]|, & \exists k \in \{1, 2, \cdots, d\} \end{cases}$$

By squaring both sides of the inequalities and summing them across all d dimensions, we have that

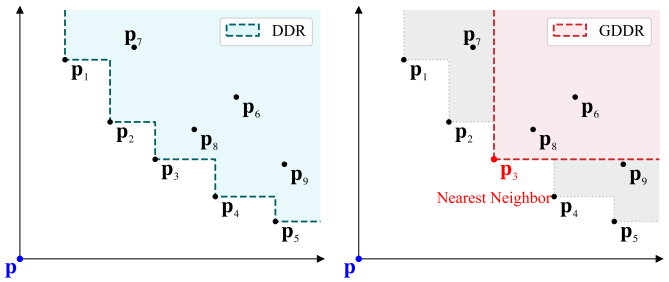
$$\sum_{j=1}^{d} (\mathbf{s}_i[j] - \mathbf{p}_i[j])^2 < \sum_{j=1}^{d} (\mathbf{o}_i[j] - \mathbf{p}_i[j])^2,$$

which is equivalent to the Euclidean distance inequality $\|\mathbf{s}_i - \mathbf{p}_i\|_2 < \|\mathbf{o}_i - \mathbf{p}_i\|_2$. However, this directly contradicts the premise that $\mathbf{o}_i$ is the global nearest neighbor with the minimum Euclidean distance to $\mathbf{p}_i$. Thus, the assumption is invalid, and $\mathbf{o}_i \in DS_{\mathbf{p}_i}$ must hold.

Since $\mathbf{o}_i \in DS_{\mathbf{p}_i}$, the single-point dominance region generated by $\mathbf{o}_i$ inherently forms a valid sub-region of DDR. Therefore, $\hat{R}_i$ is a conservative approximation of the irregular exact DDR, i.e., $\hat{R}_i \subseteq R_i$. □

GDDR Evaluation. For a query vector $\mathbf{q}$ and a data vector $\mathbf{p}_i$, given the GDDR of $\mathbf{p}_i$ represented by the $(\mathbf{p}_i, \boldsymbol{\rho}_i, \mathbf{o}_i)$, $\mathbf{q}$ is determined to fall within $\hat{R}_i$ if the distance between $\mathbf{q}$ and $\mathbf{p}_i$ is greater than or equal to the distance between $\mathbf{o}_i$ and $\mathbf{p}_i$ in all dimensions, and is strictly greater in at least one dimension.

Fig. 3 illustrates the construction process of the Dynamic Dominance Region and the Global Nearest Neighbor based Dynamic Dominance Region. All data vectors in the dataset $D = \{\mathbf{p}_i \mid 1 \le i \le 9\}$ and the pivot vector $\mathbf{p}$ are two-dimensional, which can be represented as coordinate points in a two-dimensional coordinate system. Figs. 3(a) and 3(b) depict the DDR and GDDR with respect to $\mathbf{p}$, respectively. As shown in Fig. 3(a), no data vector dominates $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4$, or $\mathbf{p}_5$; thus, these vectors constitute the dynamic skyline points of $\mathbf{p}$, denoted as $DS_{\mathbf{p}} = \{\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4, \mathbf{p}_5\}$. The DDR of $\mathbf{p}$ is formed by the union of the dominance regions of these dynamic skyline points, as indicated by the blue shaded area in the figure. In Fig. 3(b), $\mathbf{p}_3$ is the global nearest neighbor of $\mathbf{p}$, and the GDDR corresponds to the dominance region of $\mathbf{p}_3$, represented by the red shaded area. Since $\mathbf{p}_3 \in DS_{\mathbf{p}}$, the GDDR is a strict subset of the DDR, i.e., $\hat{R}_{\mathbf{p}} \subseteq R_{\mathbf{p}}$.



(a) DDR of $\mathbf{p}$. (b) GDDR of $\mathbf{p}$.

Fig. 3: An example of DDR and GDDR

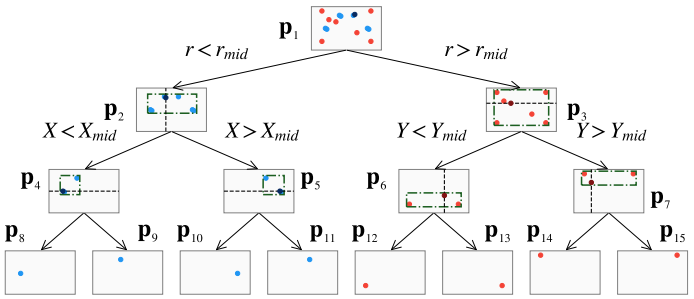


Fig. 4: The construction of Gk-tree

B. GDDR-based kd-tree Index

Since GDDR is adopted to rapidly screen candidate vectors, exhaustively verifying reverse dominance relationships still requires traversing the entire dataset. To further improve query efficiency, we propose the GDDR-based kd-tree (Gk-tree) index to save the computational cost of checking unqualified data vectors and elevate the retrieval complexity to a sub-linear scale. The details are shown as follows.

To efficiently filter irrelevant data vectors in the ciphertext domain, we integrate the GDDR with k-dimension Tree to construct the Gk-tree, denoted as $\mathcal{T}$. We first introduce the required node parameters. To support exact dominance verification, each node stores a splitting dimension α and the exact data vector $\mathbf{p}_r$ of the node. To enable GDDR-based subtree pruning, we introduce the aggregated GDDR, a consolidated GDDR region capable of filtering all data vectors within the current subtree for batch exclusion. The aggregated GDDR is defined by $(\mathbf{c}_m, \boldsymbol{\rho}_a, \mathbf{o}_a)$, where $\mathbf{c}_m$ denotes the geometric center of the Minimum Bounding Rectangle (MBR) of the current subtree, $\boldsymbol{\rho}_a$ represents the aggregated GDDR radius, and $\mathbf{o}_a$ is the aggregated boundary point that characterizes the coverage boundary of the aggregated GDDR. Consequently, each node u in $\mathcal{T}$ is formulated as a seven-element tuple, $u = \langle lch, rch, \alpha, \mathbf{c}_m, \mathbf{p}_r, \boldsymbol{\rho}_a, \mathbf{o}_a \rangle$, where $\mathbf{c}_m$ and $\mathbf{o}_a$ are jointly utilized for pruning, while $\mathbf{p}_r$ ensures accurate verification.

The specific setting of u depends on its position in $\mathcal{T}$:

1) If u is the root node, u performs the initial partition based on the GDDR radius. Specifically, it calculates the Euclidean distance r_i based on the GDDR radius and employs it as the splitting indicator. All data vectors are then sorted according to r_i. The data vectors corresponding to the median value of r_i is selected as the splitting pivot, thereby partitioning D into $u.lch$ and $u.rch$. Furthermore, $u.\mathbf{c}_m$ and $u.\boldsymbol{\rho}_a$ are derived from D and D_ρ to represent the global virtual center and the minimum aggregated GDDR radius, establishing the overall boundary $u.\mathbf{o}_a = u.\mathbf{c}_m + u.\boldsymbol{\rho}_a$.
2) If u is an internal node, u acts as both a spatial router and an actual verification candidate. Here, $u.\alpha \in$

$\{1, 2, \cdots, d\}$. $u.\mathbf{p}_r$ stores the exact coordinate of the chosen median pivot, whereas $u.\mathbf{c}_m$ stores the geometric center of the Minimum Bounding Rectangle covering its entire subtree, $u.\boldsymbol{\rho}_a$ stores the aggregated minimum GDDR radius for the subtree, and $u.\mathbf{o}_a = u.\mathbf{c}_m + u.\boldsymbol{\rho}_a$.

3) If u is a leaf node, it corresponds to a single data vector without any subtree. Here, $u.lch = u.rch = \varnothing$, $u.\alpha = \text{null}$. Both $u.\mathbf{c}_m$ and $u.\mathbf{p}_r$ equal the exact vectors, $u.\boldsymbol{\rho}_a$ is its exact GDDR radius, and $u.\mathbf{o}_a = u.\mathbf{c}_m + u.\boldsymbol{\rho}_a$ designates its boundary.

The construction of the Gk-tree index is shown in Algorithm 1. Fig. 4 shows an example of the Gk-tree index. the dataset D is first density-partitioned at the root node into two second-layer nodes based on the GDDR radius $\boldsymbol{\rho}$ with pivot $\mathbf{p}_1$. For depths greater than 2, the node is recursively split along the dimension with the maximum variance until each leaf node contains a single data point. Afterward, a bottom-up hierarchical aggregation is performed up to the second layer. A leaf node requires no aggregation and directly stores its data coordinate $\mathbf{p}_i$, GDDR radius $\boldsymbol{\rho}$, and nearest neighbor o. Conversely, a non-leaf node first computes its Minimum Bounding Rectangle (MBR) and then records the aggregated GDDR region using the MBR center $\mathbf{c}_m$, aggregated radius $\boldsymbol{\rho}_a$, and aggregated nearest neighbor $\mathbf{o}_a$.

Algorithm 1: $\mathsf{GenGkTree}(D, D_\rho)$

Input: the multi-dimensional dataset D, the GDDR radius set D_ρ
Output: the Gk-tree $\mathcal{T}$

Initialize an empty node $\mathcal{T}$;
$\mathcal{T} = \textit{BuildTree}(D, D_\rho, 0)$;
return $\mathcal{T}$;
Function $\textit{BuildTree}(D', D'_\rho, depth)$**:**
  if $|D'| = 0$ **then**
    return $\varnothing$;
  Create a new node u;
  if $depth = 0$ **then**
    $u.\alpha = d + 1$;
    Calculate the Euclidean distance $r_i = ||\boldsymbol{\rho}_i||_2$ for each $\boldsymbol{\rho}_i \in D'_\rho$;
    Sort D' and D'_ρ ascendingly based on r_i;
  else
    Set $u.\alpha$ as the dimension with the maximum variance in D';
    Sort D' and D'_ρ ascendingly based on $u.\alpha$ coordinate;
  $m = \lfloor |D'|/2 \rfloor$;
  $u.\mathbf{p}_r = D'[m]$;
  if $|D'| = 1$ **then**
    $u.lch = u.rch = \varnothing$;
    $u.\mathbf{c}_m = D'[m]$, $u.\boldsymbol{\rho}_a = D'_\rho[m]$;
    $u.\mathbf{o}_a = u.\mathbf{c}_m + u.\boldsymbol{\rho}_a$;
  else
    $u.lch = \textit{BuildTree}(D'_{<m}, D'_{\rho,<m}, depth + 1)$;
    $u.rch = \textit{BuildTree}(D'_{\geq m}, D'_{\rho,\geq m}, depth + 1)$;
    $\mathbf{p}_{min} = \min_{\mathbf{x} \in D'} \mathbf{x}$;
    $\mathbf{p}_{max} = \max_{\mathbf{x} \in D'} \mathbf{x}$;
    $u.\mathbf{c}_m = (\mathbf{p}_{min} + \mathbf{p}_{max})/2$;
    $\Delta = (\mathbf{p}_{max} - \mathbf{p}_{min})/2$;
    $u.\boldsymbol{\rho}_a = \max(u.lch.\boldsymbol{\rho}_a, u.rch.\boldsymbol{\rho}_a) + \Delta$;
    $u.\mathbf{o}_a = u.\mathbf{c}_m + u.\boldsymbol{\rho}_a$;
  return u;

C. Gk-tree based privacy-preserving reverse skyline query

Based on the proposed Gk-tree index, we give the details of the proposed Gk-tree based privacy-preserving reverse skyline query scheme (GPRSQ), which consists of the following four algorithms.

$\mathsf{GenKey}(1^\lambda) \rightarrow sk$. Given the security parameter λ, DO generates a secret key set sk and shares it with DU. sk consists of a collection of $(2d + 6) \times (2d + 6)$-dimensional random invertible matrices, defined as:

$$sk = \{G_L^{u,v,w}, G_R^{u,v,w}, H_L^{u,v,w}, H_R^{u,v,w}\}_{u,v,w=1}^{2}$$

$\mathsf{BuildIndex}(D, sk) \rightarrow \widetilde{\mathcal{T}}$. DO uses the algorithm BuildIndex to generate the encrypted Gk-tree index $\widetilde{\mathcal{T}}$ by inputting the multi-dimensional dataset D and the secret key sk. The details are described as follows:

1) For each data vector $\mathbf{p}_i \in D$, DO traverses the dataset to identify its global nearest neighbor $\mathbf{o}_i$. The GDDR radius $\boldsymbol{\rho}_i$ is then calculated by extracting the absolute distance in each dimension. Then DO obtains the complete GDDR radius set $D_\rho = \{\boldsymbol{\rho}_1, \boldsymbol{\rho}_2, \ldots, \boldsymbol{\rho}_n\}$. Subsequently, DO executes $\mathsf{GenGKTree}(D, D_\rho)$ to generate the Gk-tree $\mathcal{T}$ according to Algorithm 1.
2) For each node $u = \langle lch, rch, \alpha, \mathbf{c}_m, \mathbf{p}_r, \boldsymbol{\rho}_a, \mathbf{o}_a \rangle \in \mathcal{T}$, DO first transforms $\mathbf{o}_a$ and $\mathbf{p}_r$ into two $(2d + 6)$-dimensional matrices, denoted as G_o, H_o and G_p, H_p, respectively, where each dimension is expanded into a vector. Then DO constructs a d-dimensional all-zero vector $\mathbf{z}$ and transforms $\mathbf{c}_m$ and $\mathbf{z}$ into two sets of $(2d + 6) \times (2d + 6)$-dimensional matrices, denoted as $\mathcal{G}_c = \{G_{c,1}, G_{c,2}, \cdots, G_{c,d}\}$, $\mathcal{H}_c = \{H_{c,1}, H_{c,2}, \cdots, H_{c,d}\}$, $\mathcal{G}_z = \{G_{Z,1}, G_{Z,2}, \cdots, G_{Z,d}\}$ and $\mathcal{H}_z = \{H_{z,1}, H_{z,2}, \cdots, H_{z,d}\}$, respectively, where each dimension j is expanded into a diagonal matrix.
3) DO first randomly splits G_o, H_o, G_p and H_p into two additive matrices as follows:

$$\begin{cases} G_{o,1}[j] + G_{o,2}[j] = G_o[j] \\ H_{o,1}[j] + H_{o,2}[j] = H_o[j] \\ G_{p,1}[j] + G_{p,2}[j] = G_p[j] \\ H_{p,1}[j] + H_{p,2}[j] = H_p[j] \end{cases} \tag{2}$$

where $j \in \{1, 2, \cdots, d\}$. Then DO randomly splits each matrix in $\mathcal{G}$ and $\mathcal{H}$ into two additive matrices as follows:

$$\begin{cases} G_{c,i,1}[j] + G_{c,i,2}[j] = G_{c,i}[j] \\ H_{c,i,1}[j] + H_{c,i,2}[j] = H_{c,i}[j] \\ G_{z,i,1}[j] + G_{z,i,2}[j] = G_{z,i}[j] \\ H_{z,i,1}[j] + H_{z,i,2}[j] = H_{z,i}[j] \end{cases} \tag{3}$$

4) For each dimension $j \in \{1, 2, \cdots, d\}$ and $u, v, w \in \{1, 2\}$, DO first utilizes sk to encrypt the splitted matrices of G_o, H_o, G_p and H_p as follows:

$$\begin{aligned}
\widetilde{G}_{o,L}^{u,v,w}[j] &= (G_L^{u,v,w})^{\mathrm{T}} G_{o,u}[j] \\
\widetilde{G}_{p,L}^{u,v,w}[j] &= (G_L^{u,v,w})^{\mathrm{T}} G_{p,u}[j] \\
\widetilde{G}_{p,R}^{u,v,w}[j] &= (G_R^{u,v,w})^{-1} G_{p,w}[j] \\
\widetilde{H}_{o,L}^{u,v,w}[j] &= (H_L^{u,v,w})^{\mathrm{T}} H_{o,u}[j] \\
\widetilde{H}_{p,L}^{u,v,w}[j] &= (H_L^{u,v,w})^{\mathrm{T}} H_{p,u}[j] \\
\widetilde{H}_{p,R}^{u,v,w}[j] &= (H_R^{u,v,w})^{-1} H_{p,w}[j]
\end{aligned}$$

Thus we have the encrypted matrices sets $\mathcal{M}_{o,L} = \{\widetilde{G}_{o,L}^{u,v,w}, \widetilde{H}_{o,L}^{u,v,w}\}_{u,v,w=1}^{2}$, $\mathcal{M}_{p,L} = \{\widetilde{G}_{p,L}^{u,v,w}, \widetilde{H}_{p,L}^{u,v,w}\}_{u,v,w=1}^{2}$ and $\mathcal{M}_{p,R} = \{\widetilde{G}_{p,R}^{u,v,w}, \widetilde{H}_{p,R}^{u,v,w}\}_{u,v,w=1}^{2}$. Then DO encrypt each matrix in the splitted $\mathcal{G}$ and $\mathcal{H}$ as follows:

$$\begin{aligned}
\widetilde{G}_{c,i}^{u,v,w} &= (G_L^{u,v,w})^{-1} G_{c,i,v} G_R^{u,v,w} \\
\widetilde{H}_{c,i}^{u,v,w} &= (H_L^{u,v,w})^{-1} G_{c,i,v} H_R^{u,v,w} \\
\widetilde{G}_{z,i}^{u,v,w} &= (G_L^{u,v,w})^{-1} G_{z,i,v} G_R^{u,v,w} \\
\widetilde{H}_{z,i}^{u,v,w} &= (H_L^{u,v,w})^{-1} G_{z,i,v} H_R^{u,v,w}
\end{aligned}$$

and the encrypted matrix sets are denoted as $\mathcal{M}_{c,Q} = \{\widetilde{G}_{c,i}^{u,v,w}, \widetilde{H}_{c,i}^{u,v,w}\}_{u,v,w=1}^{2}$ and $\mathcal{M}_{z,Q} = \{\widetilde{G}_{z,i}^{u,v,w}, \widetilde{H}_{z,i}^{u,v,w}\}_{u,v,w=1}^{2}$. Finally, the ciphertext set is $\mathcal{M} = \{\mathcal{M}_{o,L}, \mathcal{M}_{p,L}, \mathcal{M}_{p,R}, \mathcal{M}_{c,Q}, \mathcal{M}_{z,Q}\}$. Consequently, the encrypted tuple is $\widetilde{u} = \langle lch, rch, \alpha, \mathcal{M} \rangle$. By applying this procedure recursively, DO generates the fully encrypted Gk-tree $\widetilde{\mathcal{T}}$.

$\mathsf{GenToken}(\mathbf{q}, sk) \rightarrow tk$. Given the query data vector $\mathbf{q}$ and the secret key sk, DU generates the query token tk. the details are described as follows:

1) DU first transforms $\mathbf{q}$ into two $(2d + 6)$-dimensional matrices, denoted as G_q, H_q. Then, DU randomly splits them into two additive matrices as follows:

$$\begin{cases} G_{q,1}[j] + G_{q,2}[j] = G_q[j] \\ H_{q,1}[j] + H_{q,2}[j] = H_q[j] \end{cases} \tag{4}$$

where $j \in \{1, 2, \cdots, d\}$.

2) For each dimension $j \in \{1, 2, \cdots, d\}$ and $u, v, w \in \{1, 2\}$, DU utilizes the secret key sk to encrypt these matrices as:

$$\begin{aligned}
\widetilde{G}_{q,L}^{u,v,w}[j] &= (G_L^{u,v,w})^{\mathrm{T}} G_{q,u}[j] \\
\widetilde{G}_{q,R}^{u,v,w}[j] &= (G_R^{u,v,w})^{-1} G_{q,w}[j] \\
\widetilde{H}_{q,L}^{u,v,w}[j] &= (H_L^{u,v,w})^{\mathrm{T}} H_{q,u}[j] \\
\widetilde{H}_{q,R}^{u,v,w}[j] &= (H_R^{u,v,w})^{-1} H_{q,w}[j]
\end{aligned}$$

where $u, v, w \in \{1, 2\}$. the encrypted matrix sets are denoted as $\mathcal{M}_{q,L} = \{\widetilde{G}_{q,L}^{u,v,w}, \widetilde{H}_{q,L}^{u,v,w}\}_{u,v,w=1}^{2}$ and $\mathcal{M}_{q,R} = \{\widetilde{G}_{q,R}^{u,v,w}, \widetilde{H}_{q,R}^{u,v,w}\}_{u,v,w=1}^{2}$. Finally, DU generates the query token $tk = \{\mathcal{M}_{q,L}, \mathcal{M}_{q,R}\}$ and transmits tk to CS.

$\mathsf{RSQuery}(\widetilde{\mathcal{T}}, tk) \rightarrow RS_{\mathbf{q}}$. Once receiving a query token tk from DU, CS executes the privacy-preserving reverse skyline query processing, which includes the following two phases:

1) **Pruning Phase:** CS executes Algorithm 3 to filter unqualified objects and obtain the candidate set C. By performing $\mathsf{MatEval}(\cdot)$, the algorithm determines whether the query data vector $\mathbf{q}$ falls within the aggregated GDDR of the current node. Consequently, subtrees completely dominated by $\mathbf{q}$ are pruned, and the candidate set C is generated.
2) **Verification Phase:** For each candidate $\mathbf{c}_i \in C$, CS runs Algorithm 4 to determine the exact dynamic dominance relationships. By executing $\mathsf{MatEval}(\cdot)$, the CS evaluates whether any data vector $\mathbf{p}_r$ dynamically dominates $\mathbf{q}$ relative to $\mathbf{c}_i$. By leveraging the spatial structure of the Gk-tree to skip irrelevant branches, the final reverse skyline query result $RS_{\mathbf{q}}$ is determined.

Algorithm 2: $\mathsf{MatEval}(\mathcal{M}_L, \mathcal{M}_Q, \mathcal{M}_R, j)$

Input: the encrypted matrix sets $\mathcal{M}_L, \mathcal{M}_Q, \mathcal{M}_R$, the dimension index j
Output: the result res

$v_L \leftarrow \{(\mathcal{M}_L.\widetilde{G}_L^{u,v,w}[j])^{\mathrm{T}}, (\mathcal{M}_L.\widetilde{H}_L^{u,v,w}[j])^{\mathrm{T}}\}_{u,v,w=1}^{2}$;
$\mathcal{S}_Q \leftarrow \{\mathcal{M}_Q.\widetilde{G}_j^{u,v,w}, \mathcal{M}_Q.\widetilde{H}_j^{u,v,w}\}_{u,v,w=1}^{2}$;
$v_R \leftarrow \{\mathcal{M}_R.\widetilde{G}_R^{u,v,w}[j], \mathcal{M}_R.\widetilde{H}_R^{u,v,w}[j]\}_{u,v,w=1}^{2}$;
$res \leftarrow \mathsf{AmeEval}(v_L, \mathcal{S}_Q, v_R)$;
return res;

Correctness. The mathematical foundation of the evaluations in Algorithm 3 and Algorithm 4 is guaranteed by $\mathsf{AmeEval}(\cdot)$. The correctness is demonstrated as follows:

$$\begin{aligned}
&\mathsf{AmeEval}(v_L, \mathcal{S}_Q, v_R) \\
&= \sum_{u,v,w=1}^{2} \Big((\widetilde{G}_L^{u,v,w}[j])^{\mathrm{T}} \widetilde{G}_j^{u,v,w} \widetilde{G}_R^{u,v,w}[j] \\
&\quad + (\widetilde{H}_L^{u,v,w}[j])^{\mathrm{T}} \widetilde{H}_j^{u,v,w} \widetilde{H}_R^{u,v,w}[j] \Big) \\
&= \sum_{u,v,w=1}^{2} \Big(G_{L,u}[j]^{\mathrm{T}} G_L^{u,v,w} (G_L^{u,v,w})^{-1} G_{j,v} \\
&\quad G_R^{u,v,w} (G_R^{u,v,w})^{-1} G_{R,w}[j] + H_{L,u}[j]^{\mathrm{T}} H_L^{u,v,w} (H_L^{u,v,w})^{-1} \\
&\quad H_{j,v} H_R^{u,v,w} (H_R^{u,v,w})^{-1} H_{R,w}[j] \Big) \\
&= \sum_{u,v,w=1}^{2} \Big(G_{L,u}[j]^{\mathrm{T}} G_{j,v} G_{R,w}[j] + H_{L,u}[j]^{\mathrm{T}} H_{j,v} H_{R,w}[j] \Big) \\
&= G_L[j]^{\mathrm{T}} G_j G_R[j] + H_L[j]^{\mathrm{T}} H_j H_R[j] \\
&= res
\end{aligned}$$

The correctness demonstrates that the secret transformation matrices (i.e., G_L, G_R, H_L, and H_R) are eliminated. This guarantees that the aggregated encrypted evaluation equals the plaintext comparison. Note that the $\mathsf{MatEval}$ subroutine used in the actual algorithms is directly derived from this $\mathsf{AmeEval}$ operator by extracting and transposing the j-th dimension

Algorithm 3: Prune($\widetilde{\mathcal{T}}, tk$)

Input: the encrypted Gk-tree $\widetilde{\mathcal{T}}$, the query token tk
Output: the candidate set C

$C \leftarrow$ *SearchPrune*($\widetilde{\mathcal{T}}.root, tk$);
return C;
Function *SearchPrune*($\widetilde{u}, tk$)**:**
  $R_{sub} \leftarrow \varnothing$;
  if $\widetilde{u} \neq \varnothing$ **then**
    $e \leftarrow 1$;
    $s \leftarrow 0$;
    for $j = 1$ **to** d **do**
      $res \leftarrow$ $\mathsf{MatEval}\big(\widetilde{u}.\mathcal{M}_{o,L}, tk.\mathcal{M}_{q,R}, \widetilde{u}.\mathcal{M}_{c,Q}, j\big)$;
      if $res < 0$ **then**
        $e \leftarrow 0$;
        break;
      if $s = 0 \wedge res > 0$ **then**
        $s \leftarrow 1$;
    if $e = 1 \wedge s = 0$ **then**
      $e \leftarrow 0$;
    if $e = 0$ **then**
      $R_{sub} \leftarrow \{\widetilde{u}\} \cup$ *SearchPrune*($\widetilde{u}.lch, tk$) $\cup$ *SearchPrune*($\widetilde{u}.rch, tk$);
  return R_{sub};

Algorithm 4: Verify($C, \widetilde{\mathcal{T}}, tk$)

Input: the candidate set C, the encrypted Gk-tree $\widetilde{\mathcal{T}}$, the query token tk
Output: the query result $RS_{\mathbf{q}}$

$RS_{\mathbf{q}} \leftarrow \varnothing$;
for ***each*** *candidate node* $\widetilde{u}_c \in C$ **do**
  if *CheckDom*($\widetilde{\mathcal{T}}.root, \widetilde{u}_c, tk$) $= 0$ **then**
    $RS_{\mathbf{q}} \leftarrow RS_{\mathbf{q}} \cup \{\widetilde{u}_c\}$;
return $RS_{\mathbf{q}}$;
Function *CheckDom*($\widetilde{u}, \widetilde{u}_c, tk$)**:**
  if $\widetilde{u} = \varnothing$ **then**
    return 0;
  if $\widetilde{u} \neq \widetilde{u}_c$ **then**
    $m \leftarrow 1$, $s \leftarrow 0$;
    for $j = 1$ **to** d **do**
      if $\mathsf{MatEval}\big(\widetilde{u}.\mathcal{M}_{p,L}, \widetilde{u}_c.\mathcal{M}_{c,Q}, tk.\mathcal{M}_{q,R}, j\big) < 0$ **then**
        $m \leftarrow 0$;
        break;
      if $s = 0 \wedge$ $\mathsf{MatEval}\big(tk.\mathcal{M}_{q,L}, \widetilde{u}_c.\mathcal{M}_{c,Q}, \widetilde{u}.\mathcal{M}_{p,R}, j\big) < 0$ **then**
        $s \leftarrow 1$;
    if $m = 1 \wedge s = 1$ **then**
      return 1;
  $val \leftarrow \mathsf{MatEval}\big(\widetilde{u}.\mathcal{M}_{p,L}, \widetilde{u}_c.\mathcal{M}_{c,Q}, tk.\mathcal{M}_{q,R}, \widetilde{u}.\alpha\big)$;
  if $\widetilde{u} = \widetilde{\mathcal{T}}.root \vee val \geq 0$ **then**
    if *CheckDom*($\widetilde{u}.lch, \widetilde{u}_c, tk$) $= 1$ **then**
      return 1;
    return *CheckDom*($\widetilde{u}.rch, \widetilde{u}_c, tk$);
  else if $\mathsf{MatEval}\big(\widetilde{u}_c.\mathcal{M}_{p,L}, \widetilde{u}.\mathcal{M}_{z,Q}, \widetilde{u}.\mathcal{M}_{p,R}, \widetilde{u}.\alpha\big) \geq 0$ **then**
    return *CheckDom*($\widetilde{u}.lch, \widetilde{u}_c, tk$);
  else
    return *CheckDom*($\widetilde{u}.rch, \widetilde{u}_c, tk$);

vectors. Thus, CS performs structural pruning and verification without recovering the actual spatial boundaries.

Time Complexity Analysis. Assuming the multi-dimensional dataset contains n points and the dimension is d, the height of the Gk-tree $\mathcal{T}$ is $O(\log n)$. During the pruning phase, CS traverses the tree and prunes unqualified subtrees, taking $O(\log n)$ time in the optimal case. Let $|C|$ be the size of the candidate set. In the verification phase, checking the dominance for each candidate requires routing through the tree, taking $O(|C| \log n)$ time. Therefore, the overall time complexity of the query processing in GkRSQ is bounded by $O(|C| \log n)$, improving the query complexity from linear to logarithmic scale compared to the exhaustive method.

VI. Security Analysis

In this section, we provide the formal security analysis of the proposed scheme. Specifically, following the definitions established in our security model, we conduct the analysis focusing on two primary aspects: IND-SCPA ciphertext indistinguishability and IND-SCPA token unlinkability.

Theorem 1: The proposed scheme GPRSQ can achieve IND-SCPA ciphertext indistinguishability under the security game.

Proof: We design the security game between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ to prove the security.

Init. $\mathcal{A}$ generates two multi-dimensional datasets $D_0 = \{p_{0,j}\}_{j\in[1,n]}$ and $D_1 = \{p_{1,j}\}_{j\in[1,n]}$ with the same dimension and sends them to $\mathcal{C}$.

Setup. Given the security parameter λ, $\mathcal{C}$ runs GenKey to generate the secret key $sk = \{G_L^{u,v,w}, G_R^{u,v,w}, H_L^{u,v,w}, H_R^{u,v,w}\}_{u,v,w=1}^{2}$, which consists of a collection of random $(2d+6) \times (2d+6)$-dimensional invertible matrices.

Phase 1. $\mathcal{A}$ first generates a series of query requests and submits to $\mathcal{C}$ in turn. Subsequently, $\mathcal{C}$ responds with a ciphertext and a token through BuildIndex and GenToken for each query request. The details are shown as follows:

- On the i-th BuildIndex request, $\mathcal{A}$ sends a multi-dimensional dataset D_i to $\mathcal{C}$. Then, $\mathcal{C}$ responds with the encrypted index $\widetilde{\mathcal{T}}_i$ through BuildIndex.
- On the i-th GenToken request, $\mathcal{A}$ sends a query request

q_i to $\mathcal{C}$, where q_i is the query data vector. Then $\mathcal{C}$ responds with the token tk_i through GenToken, where q_i is subjected to $\mathcal{L}(D_0, q_i) = \mathcal{L}(D_1, q_i)$.

Challenge. With the multi-dimensional datasets D_0 and D_1 generated in *Init*, $\mathcal{C}$ chooses a uniform bit $b \in \{0, 1\}$ and returns the encrypted index $\widetilde{\mathcal{T}}_b$ generated by BuildIndex.

Phase 2. $\mathcal{A}$ selects a series of query requests and sends them to $\mathcal{C}$, which remain bound by the same constraints as in Phase 1.

Guess. $\mathcal{A}$ outputs a guess b' of b. $\mathcal{C}$ outputs true if $b' = b$, otherwise, it outputs false.

To formally prove the indistinguishability, we analyze the ciphertext components generated by the BuildIndex algorithm. For any data vector $\mathbf{p}_i$ in the dataset D_b, the algorithm first generates the Gk-tree and then transforms each node into intermediate matrices G_o, H_o, G_p, H_p and matrix sets $\mathcal{H}_c, \mathcal{G}_z.\mathcal{H}_z$, which are then randomly splitted according to Eq. 2 and Eq. 3. Subsequently, these random matrices are encrypted using the secret invertible matrices from sk. Because the adversary $\mathcal{A}$ possesses no knowledge regarding the secret keys G_L, G_R, H_L, and H_R, the number of effective equations available to $\mathcal{A}$ is dominated by the number of unknown variables, resulting in the matrices being unable to be computed. Consequently, these unrevealed linear transformations map the uniformly distributed random matrices into a ciphertext space that remains computationally indistinguishable from random matrices. Consequently, $\mathcal{A}$ cannot distinguish the challenged ciphertext $\widetilde{\mathcal{T}}_b$ constructed from D_0 or D_1 without the secret key. Therefore, the probability that $\mathcal{A}$ correctly guesses b is bounded by

$$Adv_{\Pi_{cipher},\mathcal{A}}(\lambda) = \left| Pr[b = b'] - \frac{1}{2} \right| \leq \epsilon(\lambda),$$

where $\epsilon(\lambda)$ is a negligible function. The advantage $Adv_{\Pi_{\text{cipher}},\mathcal{A}}(\lambda)$ is negligible, proving the IND-SCPA ciphertext indistinguishability of GPSRQ. □

Theorem 2: The proposed scheme GPRSQ can achieve IND-SCPA token unlinkability under the security game.

PROOF: Similarly, we design the security game between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ to prove the security.

Init. $\mathcal{A}$ generates two query requests Q_0, Q_1 with the same dimension and sends them to $\mathcal{C}$.

Setup. Given the security parameter λ, $\mathcal{C}$ runs KeyGen to generate the secret key $sk = \{G_L^{u,v,w}, G_R^{u,v,w}, H_L^{u,v,w}, H_R^{u,v,w}\}_{u,v,w=1}^{2}$ consists of a collection of random $(2d + 6) \times (2d + 6)$-dimensional invertible matrices.

Phase 1. $\mathcal{A}$ first generates a series of query requests and submits to $\mathcal{C}$ in turn. Subsequently, $\mathcal{C}$ responds with a ciphertext and a token through BuildIndex and GenToken for each query request. The details are shown as follows:

- On the i-th BuildIndex request, $\mathcal{A}$ sends a multi-dimensional dataset D_i to $\mathcal{C}$. Then, $\mathcal{C}$ responds with the encrypted index $\widetilde{\mathcal{T}}_i$ through BuildIndex, where q_i is subjected to $\mathcal{L}(D_i, q_0) = \mathcal{L}(D_i, q_1)$.
- On the i-th GenToken request, $\mathcal{A}$ sends a query request q_i to $\mathcal{C}$, where q_i is the query data vector. Then $\mathcal{C}$ responds with the token tk_i through GenToken.

Challenge. With the query requests q_0 and q_1 generated in *Init*, $\mathcal{C}$ chooses a uniform bit $b \in \{0, 1\}$ and returns the encrypted token tk_b generated by GenToken.

Phase 2. $\mathcal{A}$ selects a series of query requests and sends them to $\mathcal{C}$, which remain bound by the same constraints as in Phase 1.

Guess. $\mathcal{A}$ outputs a guess b' of b. $\mathcal{C}$ outputs true if $b' = b$, otherwise, it outputs false.

To prove the token unlinkability, we formally analyze the generation process of the query token. During the execution of GenToken, the query vector q_b is initially mapped into intermediate dimensional matrices G_q and H_q. Similar to the index construction phase, these matrices are additively split into two random shares according to Eq. (4). Following this, the partitioned shares are encrypted using the secret keys to formulate the final token components. Since the secret key matrices remain confidential, the number of independent equations is fewer than the number of unknown variables. Therefore, it is computationally impossible for $\mathcal{A}$ tocalculate the plaintext matrices. Furthermore, the random matrix splitting mechanism ensures that even if the same query vector q_b is issued multiple times, the resulting intermediate matrices are sampled independently and uniformly at random each time. As a result, two tokens generated from identical query vectors appear as mutually independent random matrices, rendering them entirely indistinguishable from each other. Therefore, the probability that $\mathcal{A}$ correctly guesses b is bounded by

$$Adv_{\Pi_{token},\mathcal{A}}(\lambda) = \left| Pr[b = b'] - \frac{1}{2} \right| \leq \epsilon(\lambda).$$

The advantage $Adv_{\Pi_{\text{token}},\mathcal{A}}(\lambda)$ is negligible, proving the IND-SCPA token unlinkability of GPSRQ. □

VII. PERFORMANCE EVALUATION

We perform comprehensive evaluations to compare our proposed scheme GPRSQ with ePRSQ [11] and OPPRS [10]. The performance is evaluated across varying data sizes and dimensionalities. Specifically, we measure the data outsourcing time, query processing time, and storage overhead. All algorithms are implemented in Python 3.10, and the experiments are conducted on a workstation equipped with an AMD Ryzen 7 9700X processor (@ 3.80 GHz), 32 GB of RAM, and a 64-bit Windows 11 operating system. Furthermore, we employ three real-world datasets for our evaluation: the Eye State dataset, the House dataset, and the US Census dataset.

The default parameter setting is as follows, where n is the number of data records, d is the dimensionality of the data vectors. In our evaluations, when varying the data size n, the dimensionality is fixed at $d = 3$; when varying the dimensionality d, the data size is fixed at $n = 1000$.

A. Query Time Evaluation

Query time versus n. Fig. 5 shows that the query time of GPRSQ, OPPRS, and ePRSQ increases as the number of data records n grows. This is because a larger n leads to an increase in the number of vectors being evaluated, resulting in more computations and higher time costs for all

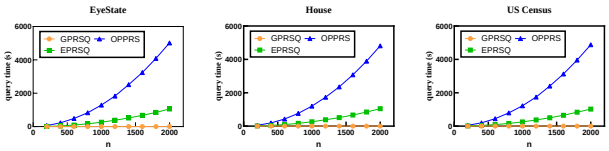


Fig. 5: Query time versus n

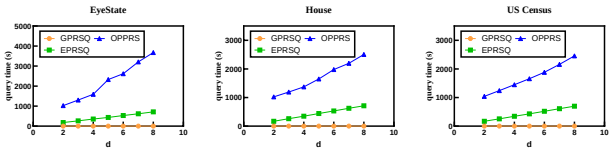


Fig. 6: Query time versus d

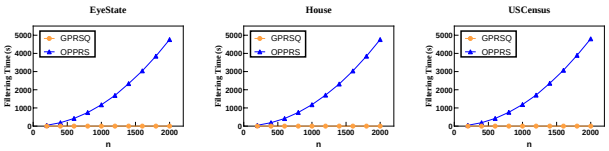


Fig. 7: Filtering Time versus n

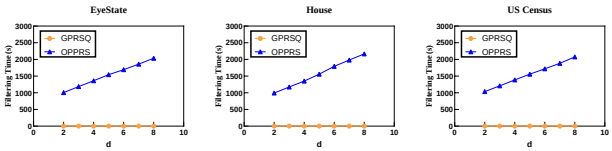


Fig. 8: Filtering Time versus d

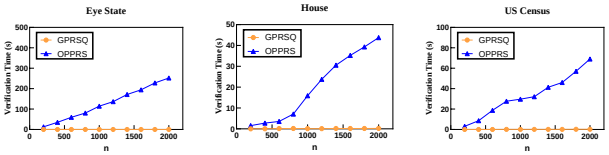


Fig. 9: Verification Time versus n

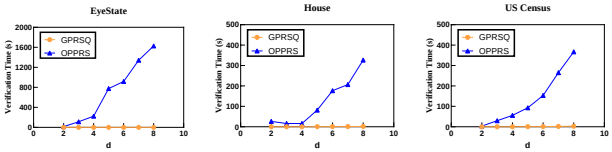


Fig. 10: Verification Time versus d

three schemes. However, as demonstrated by the experimental results, our proposed GPRSQ exhibits superior performance compared to OPPRS and ePRSQ due to its highly efficient tree-based pruning strategy. Specifically, when $n = 1000$, the query times of ePRSQ and OPPRS surge to roughly 260.7 seconds and 1292.5 seconds, respectively. In stark contrast, GPRSQ takes merely 0.033 seconds. This efficiency advantage enables GPRSQ to achieve logarithmic-scale query processing, performing better on the query phase than the linear scanning baselines.

Query time versus *d*. Fig. 6 shows that as the data dimensionality d increases, the query time of all three schemes increases simultaneously. This is because a larger d expands the dimensions of the data vectors and encrypted query tokens, which increases the computational overhead. Additionally, among the three schemes, GPRSQ exhibits a significantly slower growth rate in time cost compared to the others. For instance, when d increases from 3 to 6, the query time of OPPRS and ePRSQ drastically rises to 2619.2 seconds and 530.9 seconds, respectively. Conversely, GPRSQ remains highly stable, increasing only slightly from 0.033 seconds to 0.053 seconds. The experiment shows that GPRSQ performs exceptionally better in time cost by effectively pruning unqualified high-dimensional subtrees early.

To further analyze the internal efficiency of the query processing phase, we evaluate the filtering time and verification time of GPRSQ and compare them with the two-stage execution of OPPRS.

Filtering and verification time versus *n*. Fig. 7 and Fig. 9 show the trends of filtering time and verification time as the number of data records n increases. For OPPRS, the filtering phase requires extensive pairwise dominance comparisons across the entire dataset, leading to an high time cost. For instance, when $n = 2000$, the filtering time of OPPRS surges to 4758.3 seconds, and its verification time reaches 252.1 seconds. In stark contrast, our proposed GPRSQ leverages the Gk-tree index and GDDR-based pruning strategies to eliminate unqualified spatial regions in batches. Consequently, at the same scale of $n = 2000$, GPRSQ requires only 0.0065 seconds for filtering and 0.0317 seconds for verification. This demonstrates that GPRSQ effectively avoids the exhaustive linear scanning bottlenecks present in OPPRS.

Filtering and verification time versus *d*. Fig. 8 and Fig. 10 illustrate the impact of data dimensionality d on the filtering and verification phases. As d increases, the number of secure comparisons required to determine dominance relationships grows, which increases the computational burden. Specifically, when d increases from 3 to 6, the verification time of OPPRS increases from 113.0 seconds to 917.18 seconds, revealing a severe vulnerability to the curse of dimensionality. Conversely, GPRSQ exhibits exceptional scalability and stability in high-dimensional scenarios. Even at $d = 6$, the filtering and verification times of GPRSQ remain remarkably low at 0.0091 seconds and 0.0473 seconds, respectively. This highlights the robust pruning capability of the Gk-tree, which successfully filters out the majority of irrelevant data early in the query

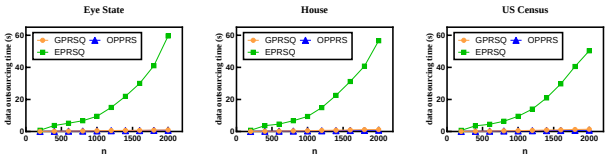


Fig. 11: data outsourcing time versus n

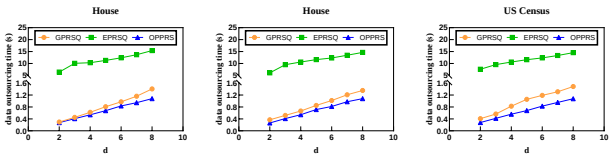


Fig. 12: data outsourcing time versus d

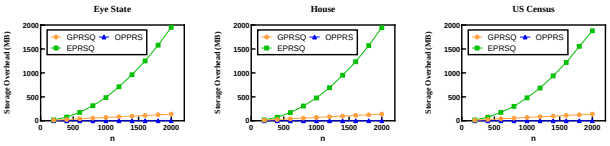


Fig. 13: Storage Overhead versus n

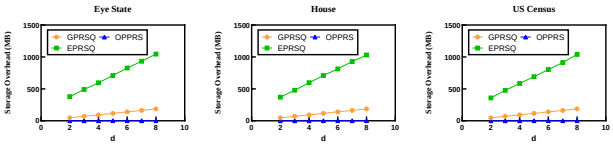


Fig. 14: Storage Overhead versus d

process, preventing the explosion of computational costs in high-dimensional spaces.

B. Data Outsourcing Time Evaluation

Data outsourcing time versus n. Fig. 11 shows that the data outsourcing time of GPRSQ, OPPRS, and ePRSQ increases as the number of data records n grows. This is because a larger n leads to an increase in the number of vectors, leading to higher computational overhead. However, our proposed GPRSQ exhibits an overwhelming efficiency advantage compared to ePRSQ. Specifically, at a scale of $n = 2000$, the data outsourcing time of ePRSQ already surges to over 43.35 seconds. In stark contrast, GPRSQ requires merely 1.00 second in total (0.36s for index generation and 0.64s for encryption), keeping the time cost on par with the OPPRS. This demonstrates that the tree-based structural design of GPRSQ successfully prevents the rapid growth of outsourcing overhead.

Data outsourcing time versus d. Fig. 12 shows that as the data dimensionality d increases, the data outsourcing time rises in all three schemes. This is because a larger d directly expands the dimensions of the data vectors, which dramatically increases the computational overhead during the setup phase. The experiment shows that our GPRSQ performs well and maintains high scalability in high-dimensional scenarios. For instance, when $n = 1000$, even as d increases to 8, the total data outsourcing time of GPRSQ remains stable at merely 1.38 seconds. Conversely, ePRSQ suffers from a severe dimensional curse, taking nearly 9.47 seconds even at a low dimensionality of $d = 3$. Consequently, GPRSQ proves to be a highly practical and efficient solution for multi-dimensional datasets.

C. Storage Evaluation

Index storage versus n. Fig. 13 shows that the index storage of GPRSQ, OPPRS, and ePRSQ increases with the number of data records n. This is because a larger n leads to more spatial points and tree nodes being encrypted and stored, resulting in higher storage requirements for all three schemes. However, our proposed GPRSQ exhibits advantage in storage efficiency compared to ePRSQ. Specifically, at a scale of $n = 2000$, the storage overhead of ePRSQ already surges to nearly 1957.44 MB. In stark contrast, GPRSQ requires merely 140.23 MB. This efficiency advantage enables GPRSQ to achieve much better scalability.

Index storage versus d. Fig. 14 shows that as the data dimensionality d increases, the index storage of all three schemes increases simultaneously. This is because a larger d expands the dimensions of the data vectors, which increases the size of the encrypted ciphertexts. Consequently, the storage costs of the three schemes rise with the expansion of d. However, the experiment shows that GPRSQ performs better in storage cost compared to ePRSQ. For instance, when the dimensionality increases to $d = 8$, the index storage of ePRSQ exceeds 2028.94 MB, whereas GPRSQ stays highly compact at merely 186.79 MB, successfully avoiding the storage explosion caused by the dimensional curse.

VIII. Conclusion

In this paper, we propose a Gk-tree based Privacy-preserving Reverse Skyline Query (GPRSQ) scheme for data marketplaces to support secure user preference matching over encrypted data. The core innovation lies in the Global Nearest Neighbor based Dynamic Dominance Region (GDDR), which transforms complex dynamic dominance boundary evaluation into efficient distance vector comparison in encrypted domains. By integrating the proposed GDDR with the kd-tree, we propose the Gk-tree index which enables pruning unqualified data vectors over encrypted vector data and significantly improves query efficiency. The proposed scheme not only protects sensitive data attributes and query information, but also prevents the cloud server from learning data distribution and query results. Formal security analysis demonstrates that the proposed scheme achieves IND-SCPA security. Extensive

experiments conducted on real world dataset demonstrate that the proposed scheme achieves high query efficiency and practical applicability in privacy-preserving data marketplaces.

ACKNOWLEGEMENT

This work was supported by the National Natural Science Foundation of China under Grant Nos. U23B2002, 62272238 and 62302232.

REFERENCES

[1] Q. Deng, Q. Zuo, Z. Li, H. Liu, and Y. Xie, "Privacy-preserving stable data trading for unknown market based on blockchain," *IEEE Trans. Mob. Comput.*, vol. 24, no. 7, pp. 5615–5631, 2025.

[2] N. Wang, S. Zhang, Z. Zhang, J. Fu, J. Liu, and R. Wang, "Block-based privacy-preserving healthcare data ranked retrieval in encrypted cloud file systems," *IEEE J. Biomed. Health Informatics*, vol. 27, no. 2, pp. 732–743, 2023.

[3] J. Wang, P. Huang, H. Zhao, Z. Zhang, B. Zhao, and D. L. Lee, "Billion-scale commodity embedding for e-commerce recommendation in alibaba," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19-23, 2018*. ACM, 2018, pp. 839–848.

[4] P. Chen, B. Xu, H. Li, W. Wang, Y. Peng, S. S. Bhowmick, X. Chen, and J. Cui, "An efficient framework for secure dynamic skyline query processing in the cloud," *Data Sci. Eng.*, vol. 10, no. 1, pp. 54–74, 2025.

[5] P. Sun, L. Wu, Z. Wang, J. Liu, J. Luo, and W. Jin, "A profit-maximizing data marketplace with differentially private federated learning under price competition," *Proc. ACM Manag. Data*, vol. 2, no. 4, pp. 191:1–191:27, 2024.

[6] X. Fang, H. Cai, B. Sheng, J. Li, J. Zhou, H. Huang, M. Ye, and F. Xiao, "A blockchain-based secure and fair online incentive mechanism for crowdsensed data trading," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 9372–9386, 2025.

[7] H. Mahmoud, M. N. Mahmoud, and A. Alsharif, "Trusttrade: A trustworthy iot data marketplace with post-trading accountability," *IEEE Internet Things J.*, vol. 12, no. 24, pp. 55 297–55 315, 2025.

[8] J. Liu, J. Yang, L. Xiong, J. Pei, J. Luo, Y. Guo, S. Ma, and C. Fan, "Skyline diagram: Efficient space partitioning for skyline queries," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 1, pp. 271–286, 2021.

[9] X. Miao, Y. Wu, J. Peng, Y. Gao, and J. Yin, "Efficient and effective cardinality estimation for skyline family," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 104:1–104:21, 2023.

[10] S. Zhang, R. Lu, and Y. Zheng, "Towards privacy-preserving online medical monitoring with reverse skyline query," in *IEEE Global Communications Conference, GLOBECOM 2021, Madrid, Spain, December 7-11, 2021*. IEEE, 2021, pp. 1–6.

[11] S. Zhang, S. Ray, R. Lu, Y. Guan, Y. Zheng, and J. Shao, "Toward privacy-preserving aggregate reverse skyline query with strong security," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2538–2552, 2022.

[12] Y. Peng, X. Li, K. Gu, J. Chen, S. K. Das, and X. Zhang, "Achieving efficient and privacy-preserving reverse skyline query over single cloud," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 1, pp. 29–44, 2025.

[13] S. Börzsönyi, D. Kossmann, and K. Stocker, "The skyline operator," in *Proceedings of the 17th International Conference on Data Engineering (ICDE)*. IEEE, 2001, pp. 421–430.

[14] D. Papadias, Y. Tao, G. Fu, and B. Seeger, "An optimal and progressive algorithm for skyline queries," in *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, 2003, pp. 467–478.

[15] E. Dellis and B. Seeger, "Efficient computation of reverse skyline queries," in *Proceedings of the 33rd international conference on Very large data bases (VLDB)*, 2007, pp. 291–302.

[16] X. Miao, Y. Wu, J. Peng, Y. Gao, and J. Yin, "Efficient and effective cardinality estimation for skyline family," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, pp. 1–25, 2023.

[17] K. Mouratidis, Y. Li, and B. Tang, "Marrying top-k with skyline queries: Operators with relaxed preference input and controllable output size," *ACM Transactions on Database Systems (TODS)*, 2025.

[18] P. Ciaccia and D. Martinenghi, "Directional queries: Making top-k queries more effective in discovering relevant results," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–26, 2024.

[19] ——, "Flexible skylines: Dominance for arbitrary sets of monotone functions," *ACM Transactions on Database Systems (TODS)*, vol. 45, no. 4, pp. 1–44, 2020.

[20] C. Liu, J. Zheng, Y. Zhao, B. Zheng, X. Lin, and H. Chen, "Skyline diagram: Efficient space partitioning for skyline queries," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 33, no. 1, pp. 271–286, 2021.

[21] A. Hidayat, M. A. Cheema, X. Lin, W. Zhang, and Y. Zhang, "Continuous monitoring of moving skyline and top-k queries," *The VLDB Journal*, vol. 31, no. 3, pp. 459–482, 2022.

[22] E. Dellis and B. Seeger, "Efficient computation of reverse skyline queries," in *Proceedings of the 33rd International Conference on Very Large Data Bases, University of Vienna, Austria, September 23-27, 2007*. ACM, 2007, pp. 291–302.

[23] H. Zhu, X. Li, Q. Liu, and Z. Xu, "Top-k dominating queries on skyline groups," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 7, pp. 1431–1444, 2020.

[24] Z. Bian, X. Yan, J. Zhang, M. L. Yiu, and B. Tang, "QSRP: efficient reverse k-ranks query processing on high-dimensional embeddings," in *40th IEEE International Conference on Data Engineering, ICDE Utrecht, The Netherlands, May 13-16*. IEEE, 2024, pp. 4614–4627.

[25] S. Li, X. Zhang, and L. Zhang, "Research on multiple enhanced k combination reverse skyline query method," *J. Big Data*, vol. 12, no. 1, p. 19, 2025.

[26] Y. Zheng, R. Lu, B. Li, J. Shao, H. Yang, and K. R. Choo, "Efficient privacy-preserving data merging and skyline computation over multi-source encrypted data," *Inf. Sci.*, vol. 498, pp. 91–105, 2019.

[27] X. Liu, K. R. Choo, R. H. Deng, Y. Yang, and Y. Zhang, "PUSC: privacy-preserving user-centric skyline computation over multiple encrypted domains," in *17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications / 12th IEEE International Conference On Big Data Science And Engineering, TrustCom/BigDataSE 2018, New York, NY, USA, August 1-3, 2018*. IEEE, 2018, pp. 958–963.

[28] S. Zhang, S. Ray, R. Lu, Y. Zheng, Y. Guan, and J. Shao, "Towards efficient and privacy-preserving user-defined skyline query over single cloud," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 2, pp. 1319–1334, 2023.

[29] ——, "Towards efficient and privacy-preserving interval skyline queries over time series data," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 2, pp. 1348–1363, 2023.

[30] J. Liu, J. Yang, L. Xiong, and J. Pei, "Secure skyline queries on cloud platform," in *33rd IEEE International Conference on Data Engineering, ICDE 2017, San Diego, CA, USA, April 19-22, 2017*. IEEE Computer Society, 2017, pp. 633–644.

[31] W. Wang, H. Li, Y. Peng, S. S. Bhowmick, P. Chen, X. Chen, and J. Cui, "SCALE: an efficient framework for secure dynamic skyline query processing in the cloud," in *Database Systems for Advanced Applications - 25th International Conference, DASFAA 2020, Jeju, South Korea, September 24-27, 2020, Proceedings, Part III*, ser. Lecture Notes in Computer Science. Springer, 2020, pp. 288–305.

[32] S. Zeighami, G. Ghinita, and C. Shahabi, "Secure dynamic skyline queries using result materialization," in *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 2021, pp. 157–168.

[33] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Commun. ACM*, vol. 18, no. 9, pp. 509–517, 1975.

[34] Y. Zheng, R. Lu, S. Zhang, J. Shao, and H. Zhu, "Achieving practical and privacy-preserving knn query over encrypted data," *IEEE Trans. Dependable Secur. Comput.*, vol. 21, no. 6, pp. 5479–5492, 2024.

[35] B. Wang, M. Li, and H. Wang, "Geometric range search on encrypted spatial data," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 4, pp. 704–719, 2016.

[36] D. Cash and S. Tessaro, "The locality of searchable symmetric encryption," in *EUROCRYPT 2014, 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, vol. 8441, 2014, pp. 351–368.

[37] D. Cash, S. Jarecki, C. S. Jutla, H. Krawczyk, M. Rosu, and M. Steiner, "Highly-scalable searchable symmetric encryption with support for boolean queries," in *CRYPTO 2013, 33rd Annual Cryptology Conference*, vol. 8042, 2013, pp. 353–373.